8.7.1 LiveServerTestCase for Selenium

There are several UI testing tools out there for websites, with one of the most popular being Selenium (https://www.selenium.dev/). There are entire books on using Selenium; here, I'll just give you a quick taste. Selenium integrates with web browsers and drives them the same way a user does. There are two general modes: click-and-record and API-based. Click-and-record uses a tool that tracks what you do to a web page, allowing you to play it back again as part of your tests. Or you can write scripts that use the API instead. These scripts include code that searches for tags in the HTML and interacts with them, by, for example, clicking links.

Click-and-record is faster to get going and requires less training. If you can learn to hit the Record button and use the web page you want to test, you're pretty much there. This method does tend to be a little more fragile. Small changes to the page may cause the script to break.

The API method requires more work and a bit of careful planning when you write your HTML. The easiest way to find an item in a web page is to search by its ID. Of course, you may not have put IDs on all of your tags. There are other ways to search, but they tend to suffer from the same fragility as the click-and-record method. I generally prefer to use the API and write a testable web page with IDs in all the tags.

Django's `TestCase` class calls views directly, meaning any JavaScript doesn't get called. If you want to test the page the way a browser sees it, you need a server. You could use the development server, but then, you have to make sure the state of your site is correct before running the Selenium script. Instead, you use `django.test.LiveServerTestCase` or its sibling `StaticLiveServerTestCase`. These test classes cause Django to run the development server during testing. As both classes inherit from `TestCase`, you can still use `.setUp()` to create any required data and then call the Selenium API from the test methods. As the name implies, the `StaticLiveServerTestCase` also runs a static server, so your static files, like CSS and JavaScript, get served by the test harness. For more information on writing a Selenium test with Django, see the sample script in the documentation: https://mng.bz/wxx7.

8.7.2 Third-party helpers

This section could also be titled *Shameless Plug*. I'd like to point you at one of my own third-party libraries: Django Awl (https://github.com/cltrudeau/django-awl). This library is an odds-and-ends collection of stuff I tend to use in every Django project. I kept rewriting the same code for each new project and figured it was better to have a library instead. The `awl.waelsteng` module contains helpers for testing. There are tools for testing your Django Admin customizations, a class that acts like a request object in case you want to call a view without using the URL routing feature, and a custom test runner.

The Python `unittest` test runner (and its Django child) accepts arguments on the command line, allowing you to run a subset of the tests. For example, if you wished to run only the `test_musician_view()` method you could: